

Vulnerabilidades del OWASP Top 10

Informe de Ciberseguridad

Alumno: Antonio L. Rodríguez Pérez

Fecha: 12/03/2026

1. Introducción

La seguridad en aplicaciones web es un aspecto fundamental dentro del ámbito de la ciberseguridad moderna. En la actualidad, una gran parte de los servicios utilizados por organizaciones, empresas y usuarios se ofrecen a través de aplicaciones web. Estas aplicaciones gestionan información sensible como credenciales de acceso, datos personales, información financiera o información corporativa crítica.

Sin embargo, cuando las aplicaciones web no se desarrollan siguiendo buenas prácticas de seguridad, pueden contener vulnerabilidades que permiten a los atacantes comprometer la confidencialidad, integridad o disponibilidad de los sistemas. Para ayudar a identificar y mitigar estos riesgos, la organización **OWASP (Open Web Application Security Project)** publica periódicamente el listado conocido como **OWASP Top 10**, que identifica las vulnerabilidades más críticas y frecuentes presentes en aplicaciones web.

En esta práctica se ha utilizado la aplicación vulnerable **bWAPP (Buggy Web Application)**, una plataforma diseñada específicamente para el aprendizaje de seguridad informática. Esta aplicación contiene múltiples vulnerabilidades intencionadas que permiten simular ataques reales en un entorno controlado.

A lo largo de esta práctica se han analizado y explotado diferentes vulnerabilidades incluidas en el OWASP Top 10 con el objetivo de comprender su funcionamiento, evaluar los riesgos asociados y analizar las posibles medidas de mitigación.

2. Objetivos de la práctica

Los principales objetivos de esta práctica son:

- Comprender el funcionamiento de las vulnerabilidades más comunes en aplicaciones web.
- Analizar cómo pueden ser explotadas por un atacante.
- Identificar los riesgos asociados a cada vulnerabilidad.
- Aprender técnicas básicas de auditoría de seguridad web.
- Comprender la importancia de implementar mecanismos de seguridad adecuados.
- Conocer las medidas de mitigación necesarias para prevenir estos ataques.

3. Entorno de laboratorio

Para la realización de esta práctica se ha utilizado un entorno de laboratorio virtualizado que permite realizar pruebas de seguridad sin afectar a sistemas reales.

El entorno utilizado está compuesto por:

- BeeBox:** máquina virtual que contiene la aplicación vulnerable bWAPP.
- bWAPP:** aplicación web vulnerable diseñada para el aprendizaje de seguridad informática.
- Servidor Apache:** encargado de servir la aplicación web.
- Base de datos MySQL:** utilizada para almacenar los datos de la aplicación.
- Navegador web:** utilizado para interactuar con la aplicación.
- Terminal Linux:** utilizada para acceder a la base de datos y ejecutar comandos.
-

La aplicación se accedió a través del navegador web utilizando la dirección:

<http://localhost/bWAPP>

Este entorno permite realizar pruebas de seguridad y simular ataques reales de forma controlada y segura

4. Análisis de vulnerabilidades

4.1 Broken Access Control – Insecure Direct Object Reference (DOR)

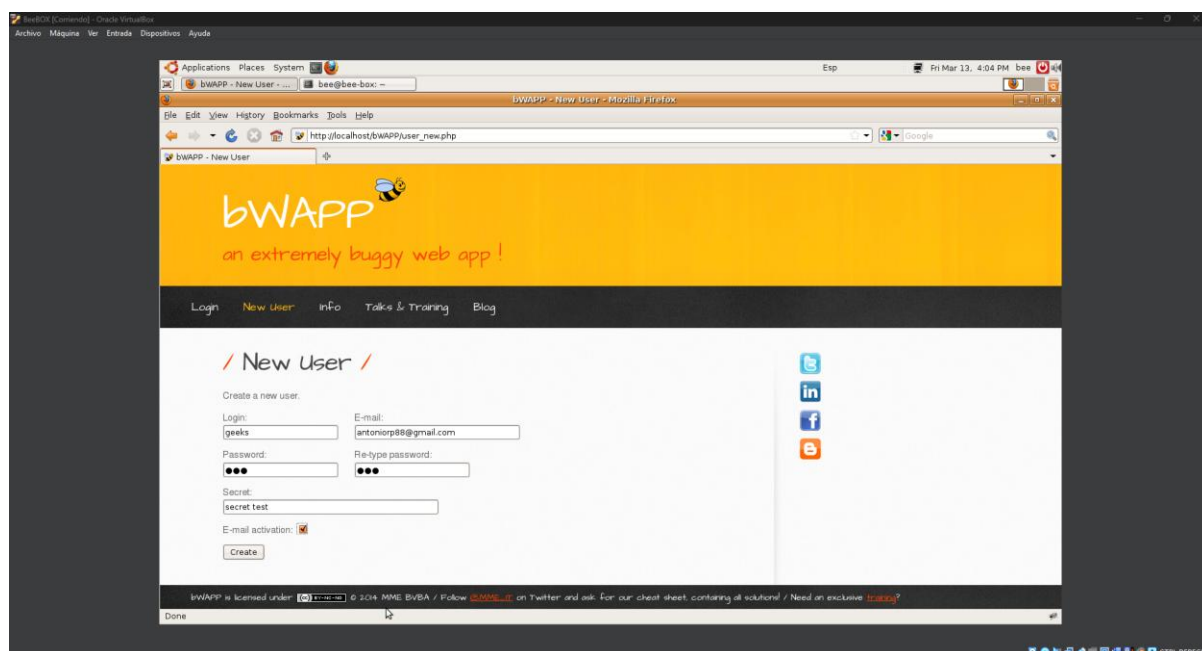
Descripción:

La vulnerabilidad **Broken Access Control** se produce cuando una aplicación no implementa correctamente los mecanismos de control de acceso. Esto permite que los usuarios accedan a recursos o funcionalidades para los que no tienen autorización.

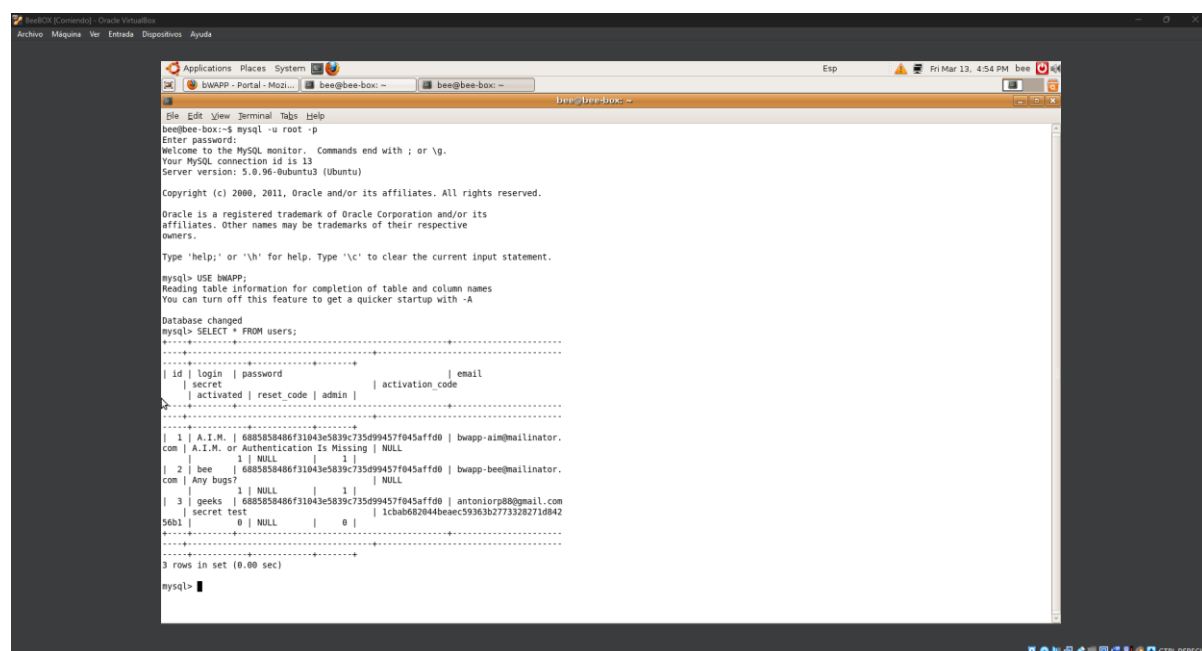
Uno de los casos más comunes es **Insecure Direct Object Reference (IDOR)**, donde la aplicación utiliza identificadores directos para acceder a recursos internos sin verificar si el usuario tiene permisos adecuados.

Procedimiento

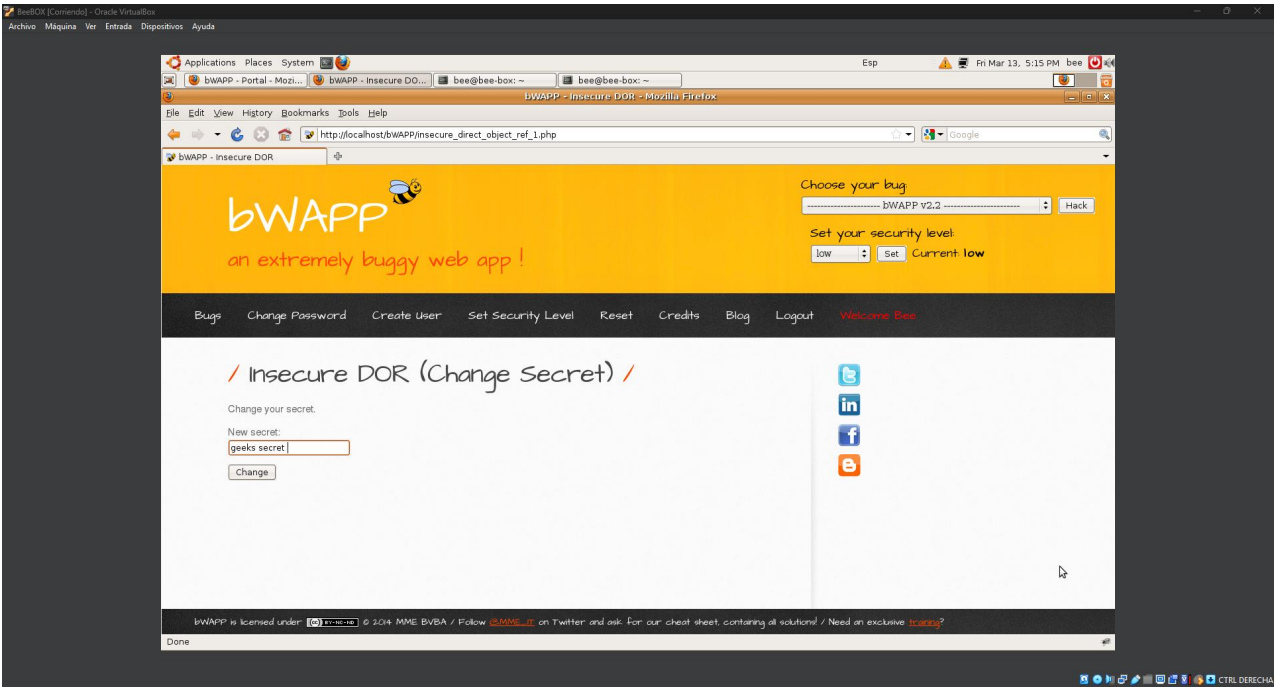
- Primero se creó un nuevo usuario en la aplicación llamado **geeks**.



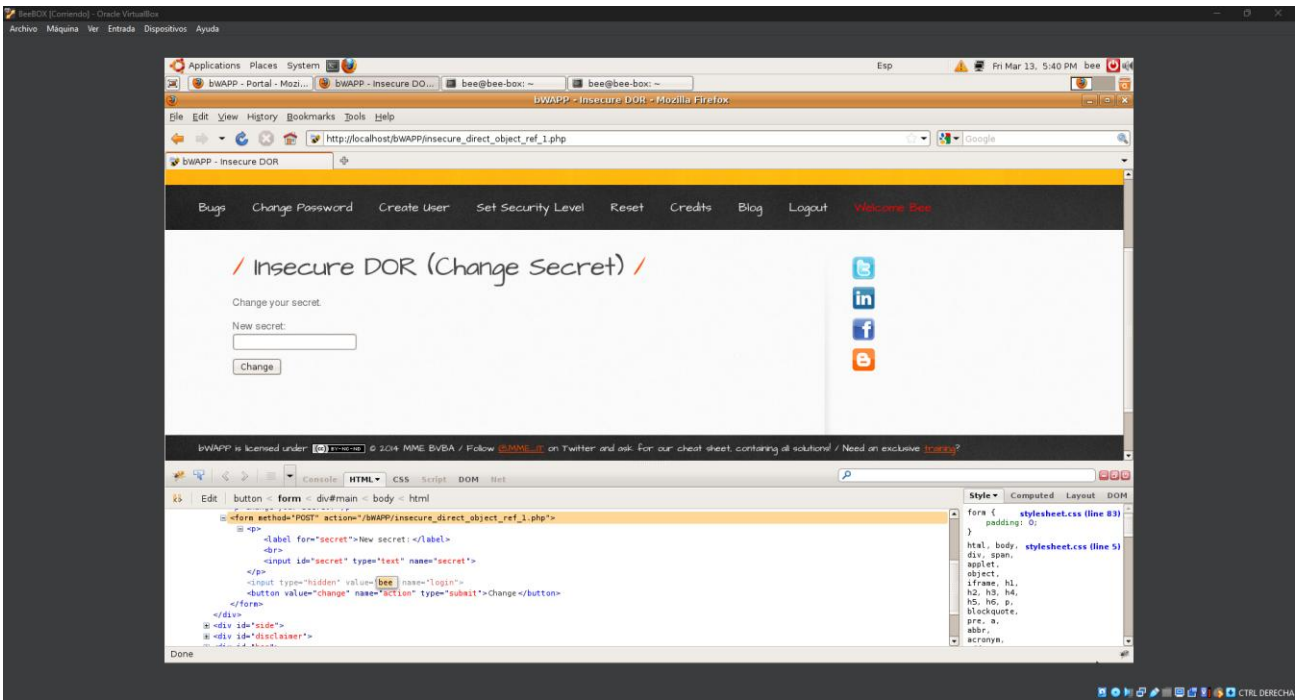
- Posteriormente se accedió a la base de datos MySQL para comprobar que el usuario había sido creado correctamente.



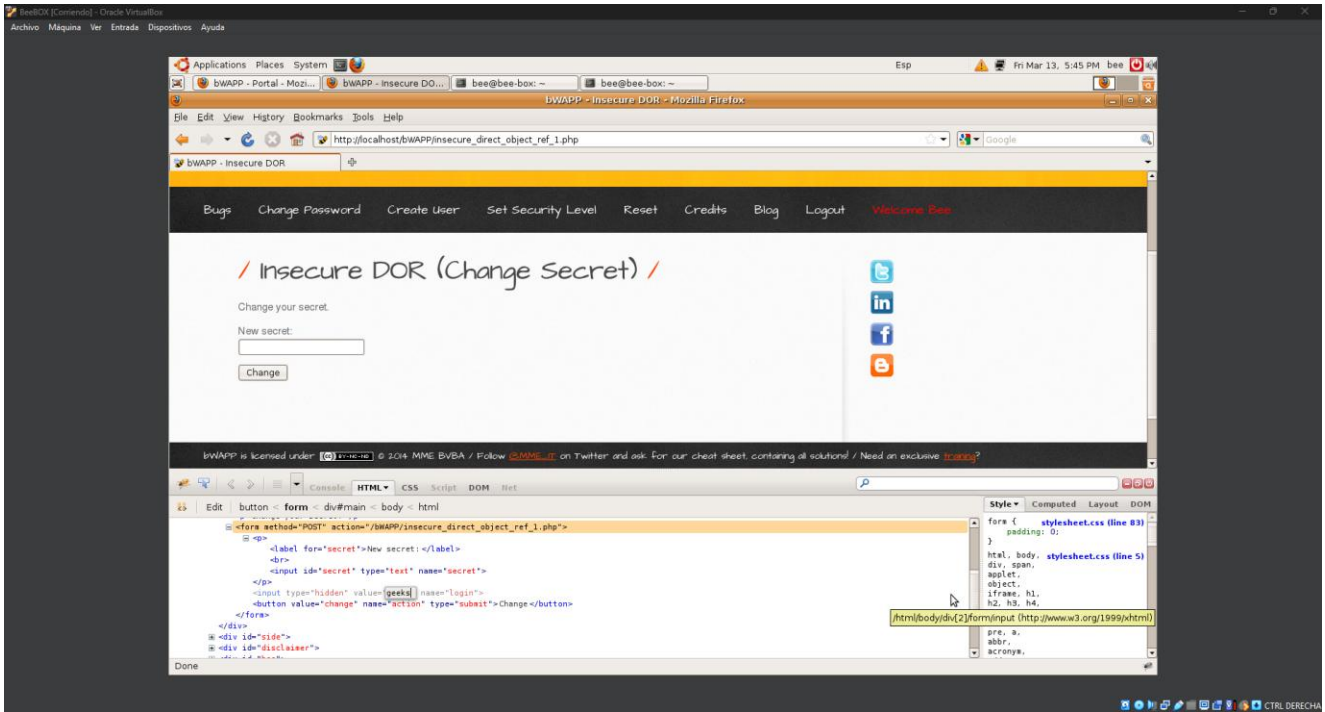
- Después se accedió a la vulnerabilidad **Insecure DOR (Change Secret)** y se inspeccionó el formulario HTML.



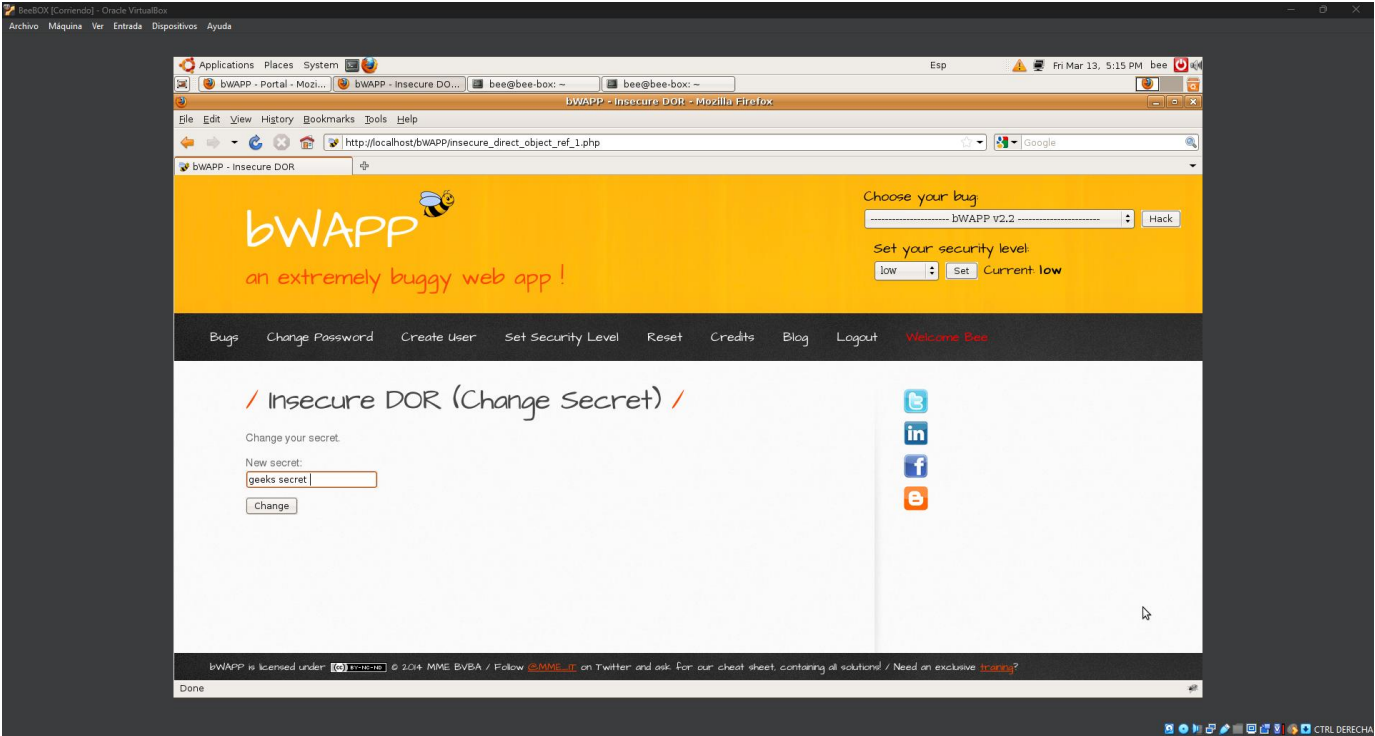
- Se localizó un campo oculto que contenía el nombre del usuario actual.



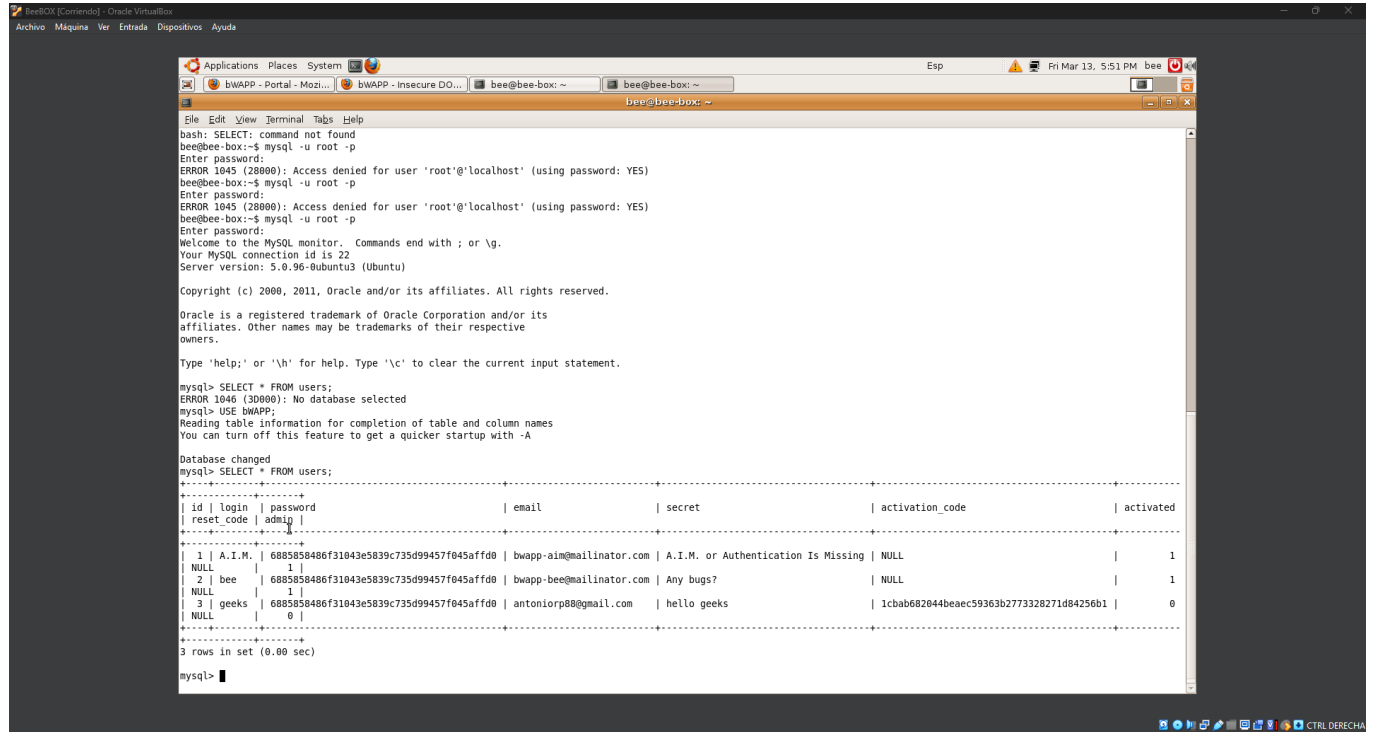
- Se modificó manualmente este valor cambiándolo por **geeks**.



- Finalmente se envió el formulario cambiando el secret del usuario.



- Se verificó en la base de datos que el secret del usuario geeks había cambiado.



Explicación técnica

El problema ocurre porque la aplicación confía en un parámetro enviado por el cliente para identificar al usuario cuyo secret será modificado. Como el parámetro puede ser manipulado en el navegador, un atacante puede cambiar el valor y modificar datos de otros usuarios.

Riesgos

- Modificación de datos de otros usuarios.
- Acceso no autorizado a información sensible.
- Escalada de privilegios.

Mitigación

- Implementar controles de acceso en el servidor.
- Validar siempre el usuario autenticado.
- No confiar en datos enviados por el cliente.

4.2 Broken Authentication

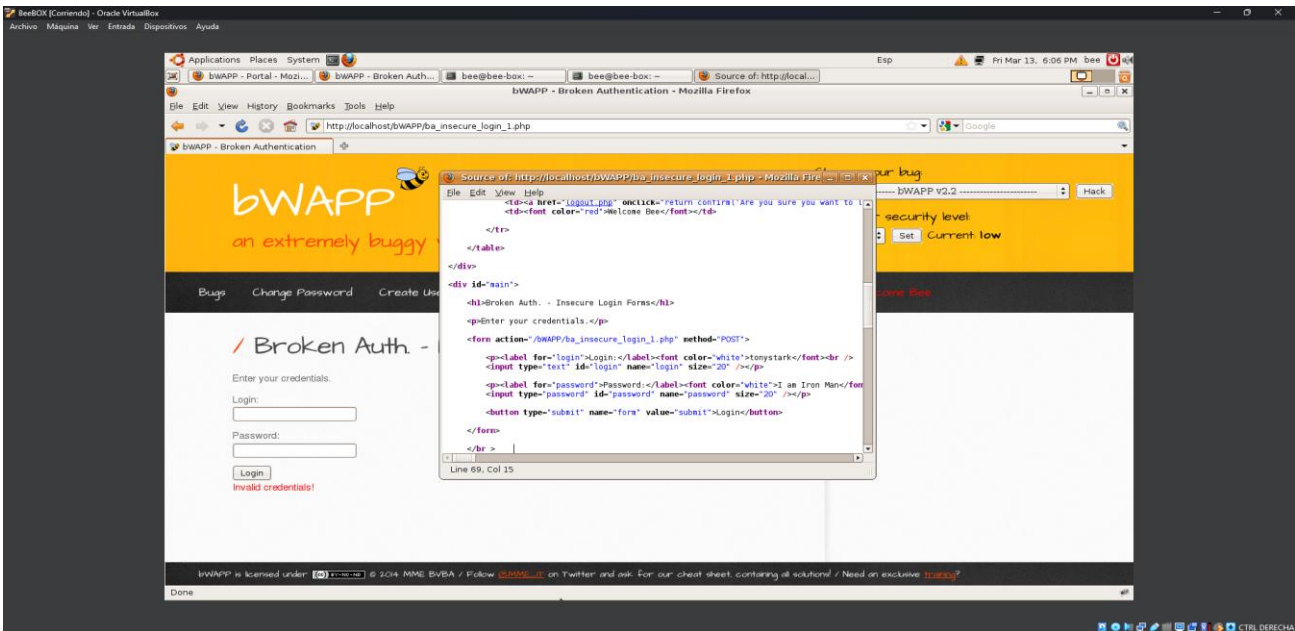
Descripción

La vulnerabilidad **Broken Authentication** ocurre cuando una aplicación gestiona incorrectamente los procesos de autenticación y sesión de usuario.

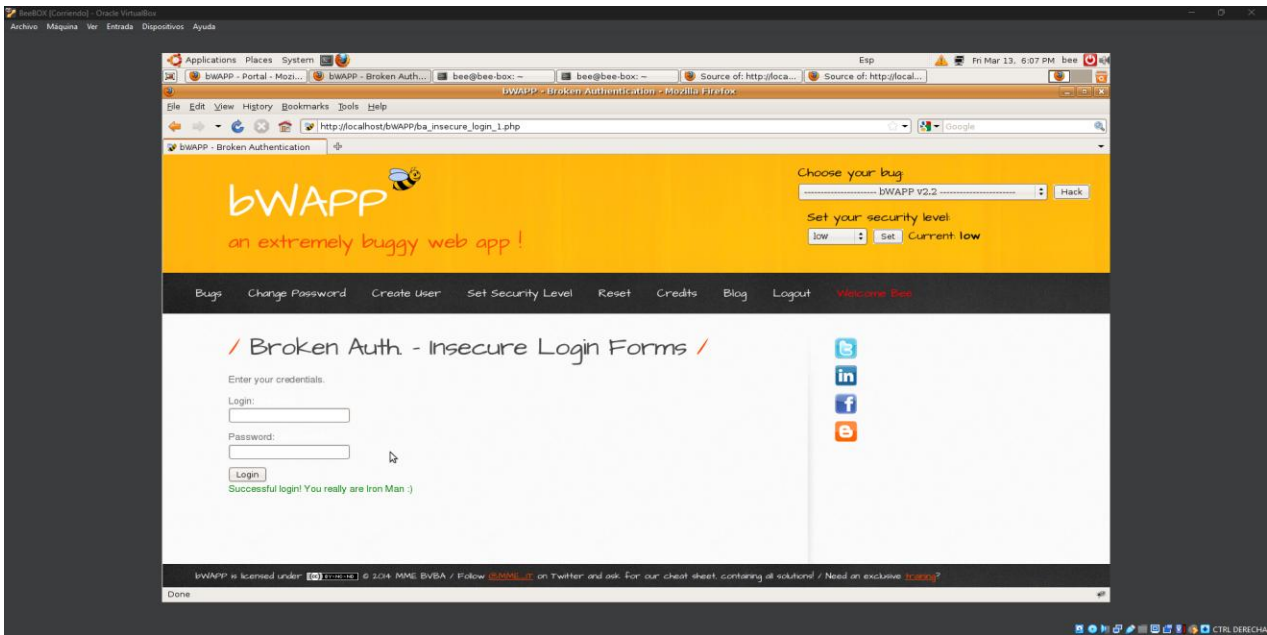
Procedimiento

Se accedió a la sección **Insecure Login Forms**.

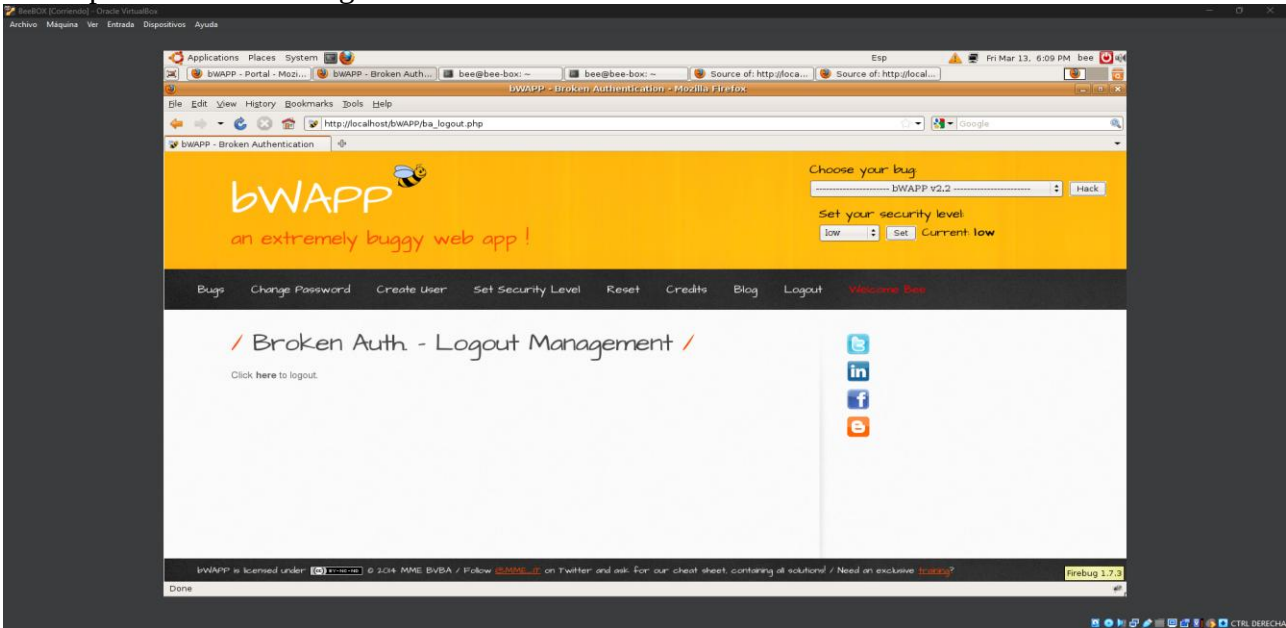
- Al revisar el código fuente de la página se encontraron credenciales incrustadas en el HTML.



- Posteriormente se inició sesión utilizando estas credenciales.



- También se comprobó el fallo en la gestión de cierre de sesión.



Explicación técnica

Las credenciales se encuentran directamente en el código fuente de la página, lo que permite a cualquier usuario obtenerlas simplemente inspeccionando el HTML.

Riesgos

- Compromiso de cuentas de usuario.
- Acceso no autorizado al sistema.
- Robo de credenciales.

Mitigación

- No almacenar credenciales en el código fuente.
- Implementar sistemas de autenticación seguros.
- Invalidar correctamente las sesiones.

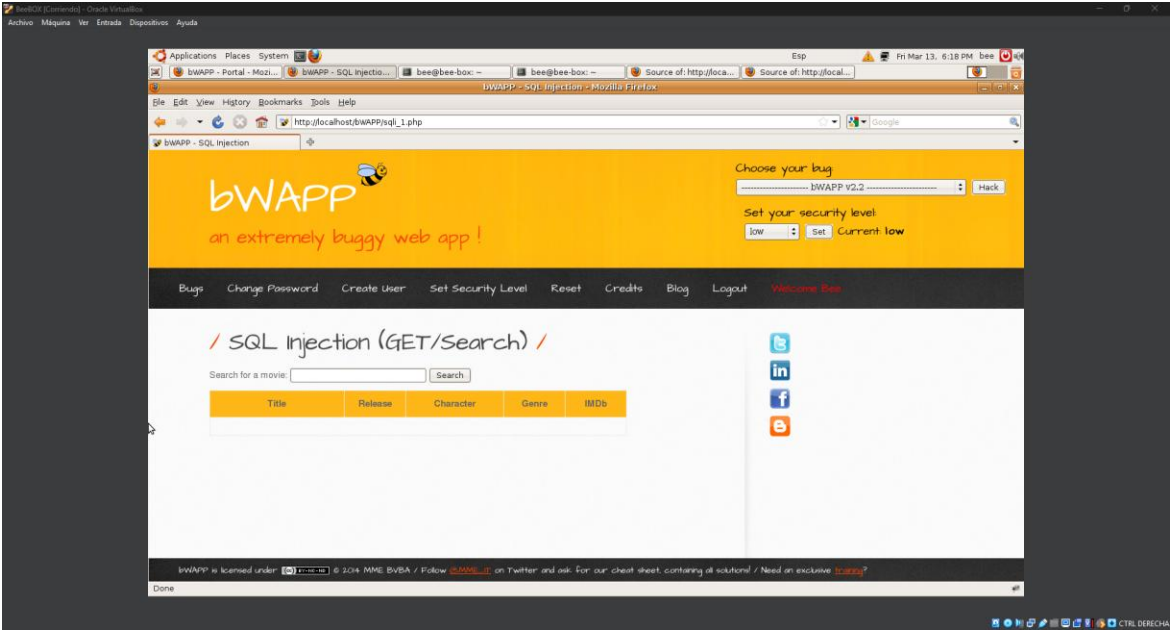
4.3 SQL Injection

Descripción

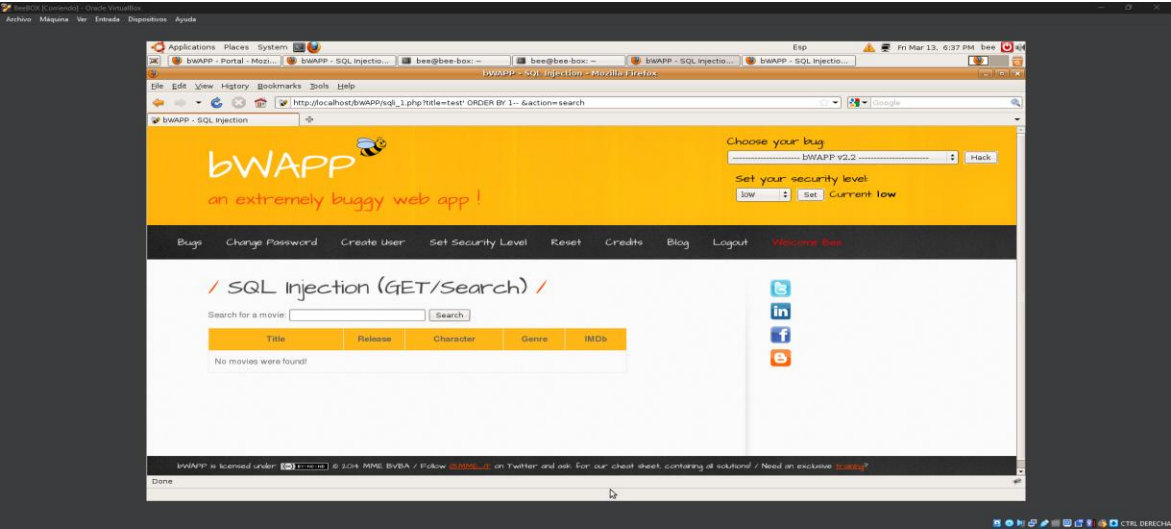
SQL Injection es una vulnerabilidad que permite a un atacante manipular consultas SQL ejecutadas por la aplicación.

Procedimiento

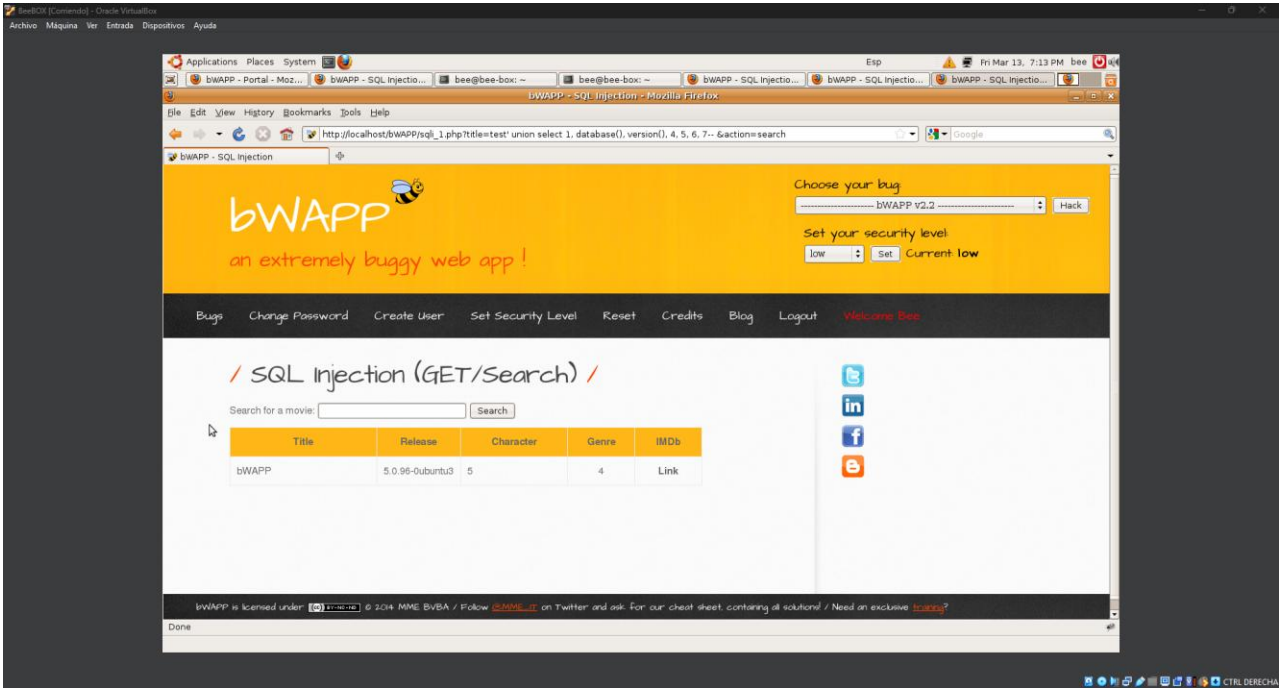
- Se accedió al formulario vulnerable de búsqueda.



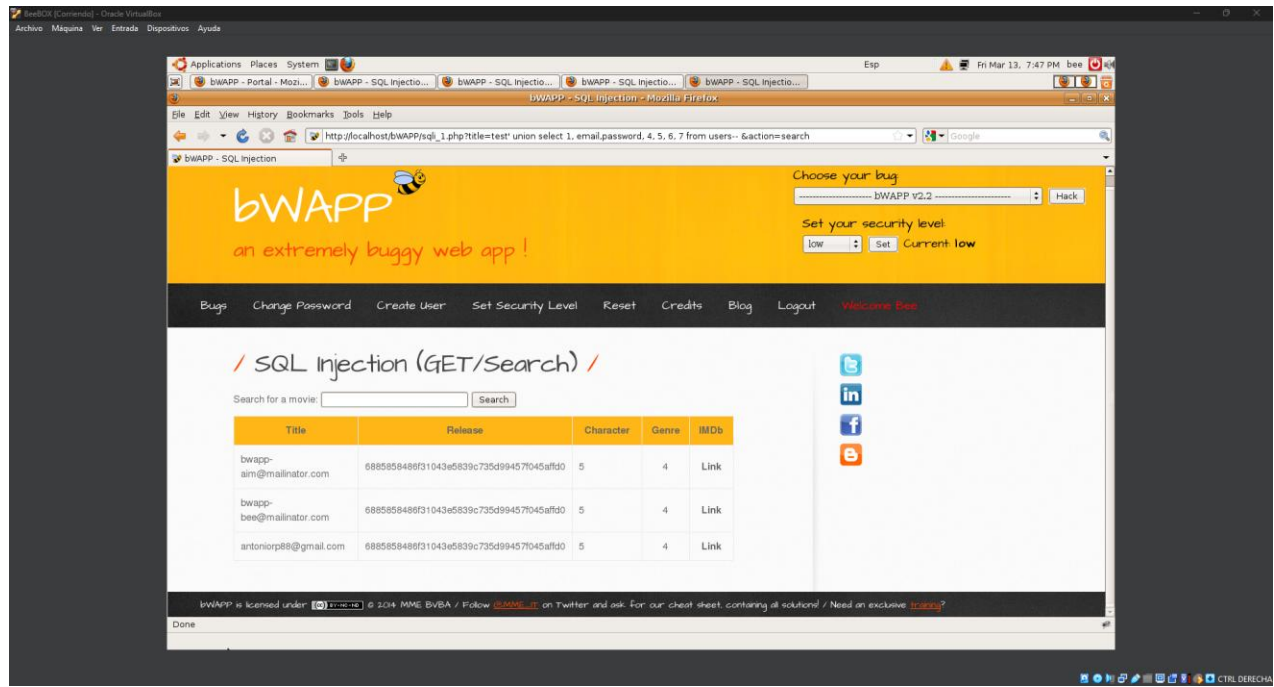
Se utilizaron diferentes consultas con **ORDER BY** para determinar el número de columnas.



- Posteriormente se utilizó **UNION SELECT** para obtener información de la base de datos.



- Finalmente se obtuvieron usuarios y contraseñas.



Explicación técnica

La aplicación inserta directamente los datos del usuario en la consulta SQL sin validación.

Riesgos

- Acceso a la base de datos.
 - Robo de información sensible.
 - Modificación o eliminación de datos.
-

Mitigación

- Uso de **consultas preparadas (Prepared Statements)**.
 - Validación de entradas.
 - Principio de mínimo privilegio en bases de datos.
-

4.4 Cross Site Scripting (XSS)

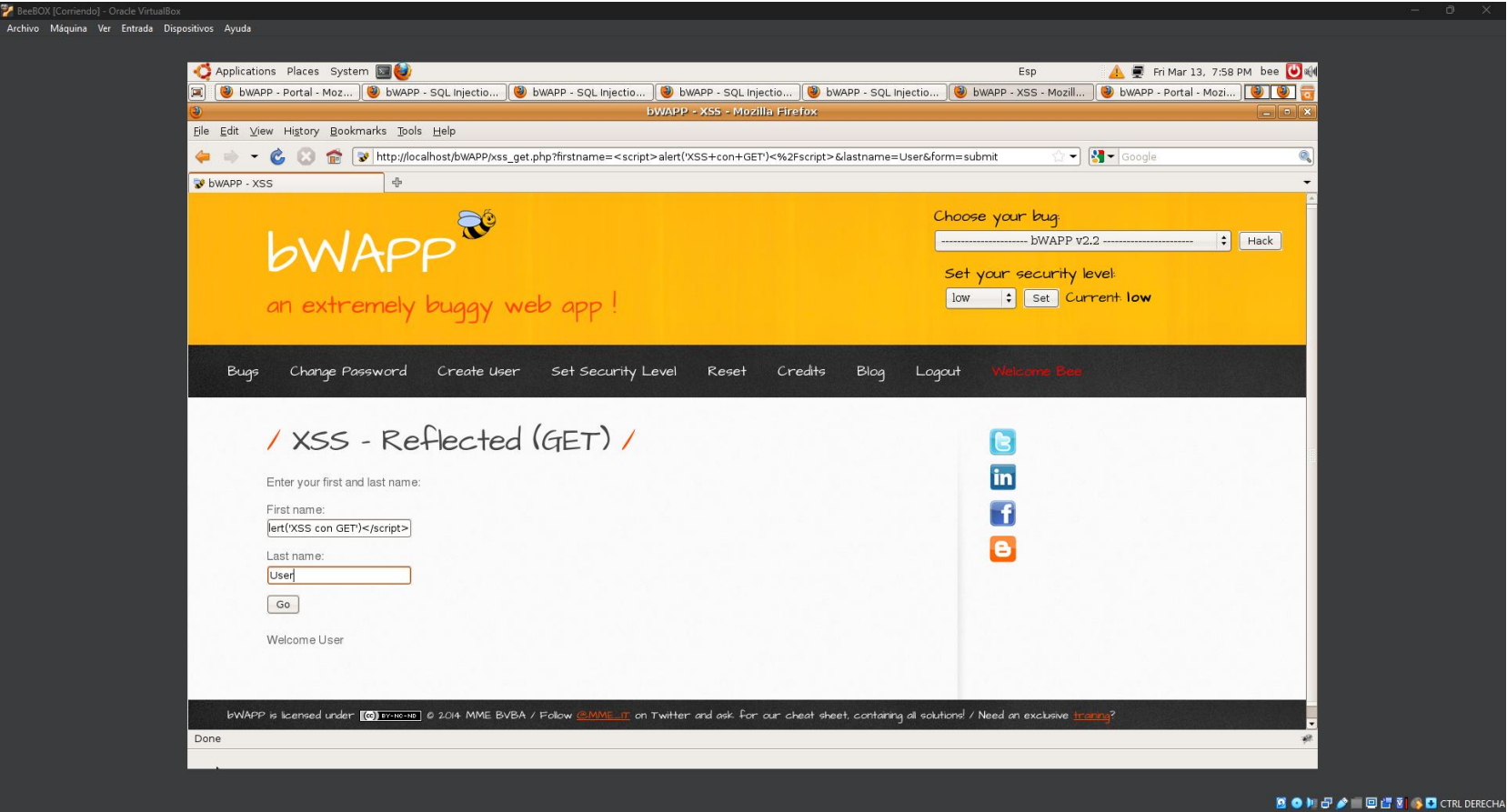
Descripción

XSS permite a un atacante inyectar código JavaScript malicioso que se ejecuta en el navegador de otros usuarios.

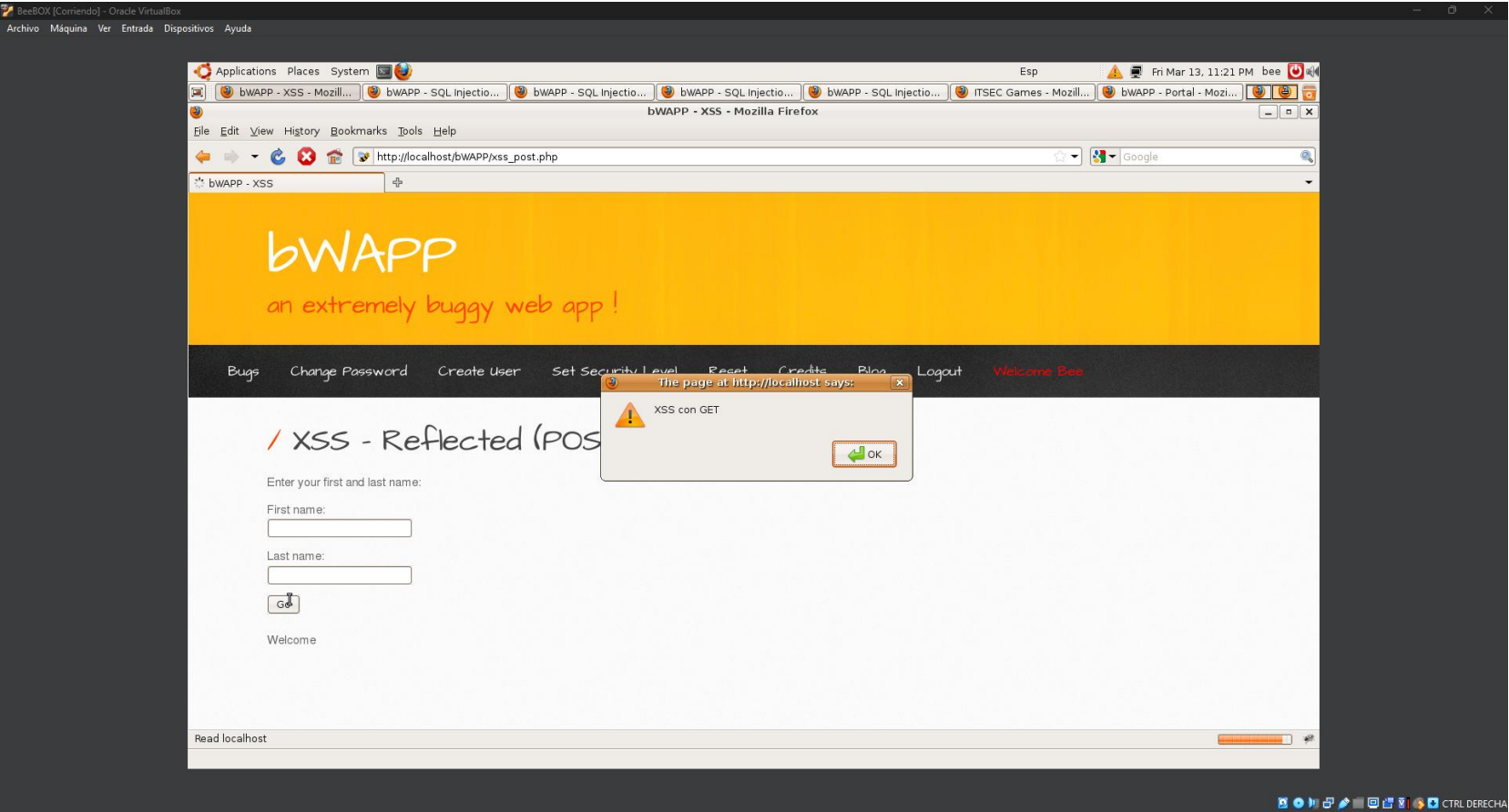
Procedimiento

- Se introdujo el siguiente script en el formulario:

```
<script>alert('XSS')</script>
```



- Al enviar el formulario el script se ejecutó.



Explicación técnica

La aplicación no valida ni sanitiza correctamente las entradas del usuario antes de mostrarlas.

Riesgos

- Robo de cookies.
 - Secuestro de sesiones.
 - Redirecciones maliciosas.
-

Mitigación

- Escapar caracteres especiales.
 - Filtrar entradas de usuario.
 - Implementar **Content Security Policy (CSP)**.
-

4.5 Local File Inclusion (LFI)

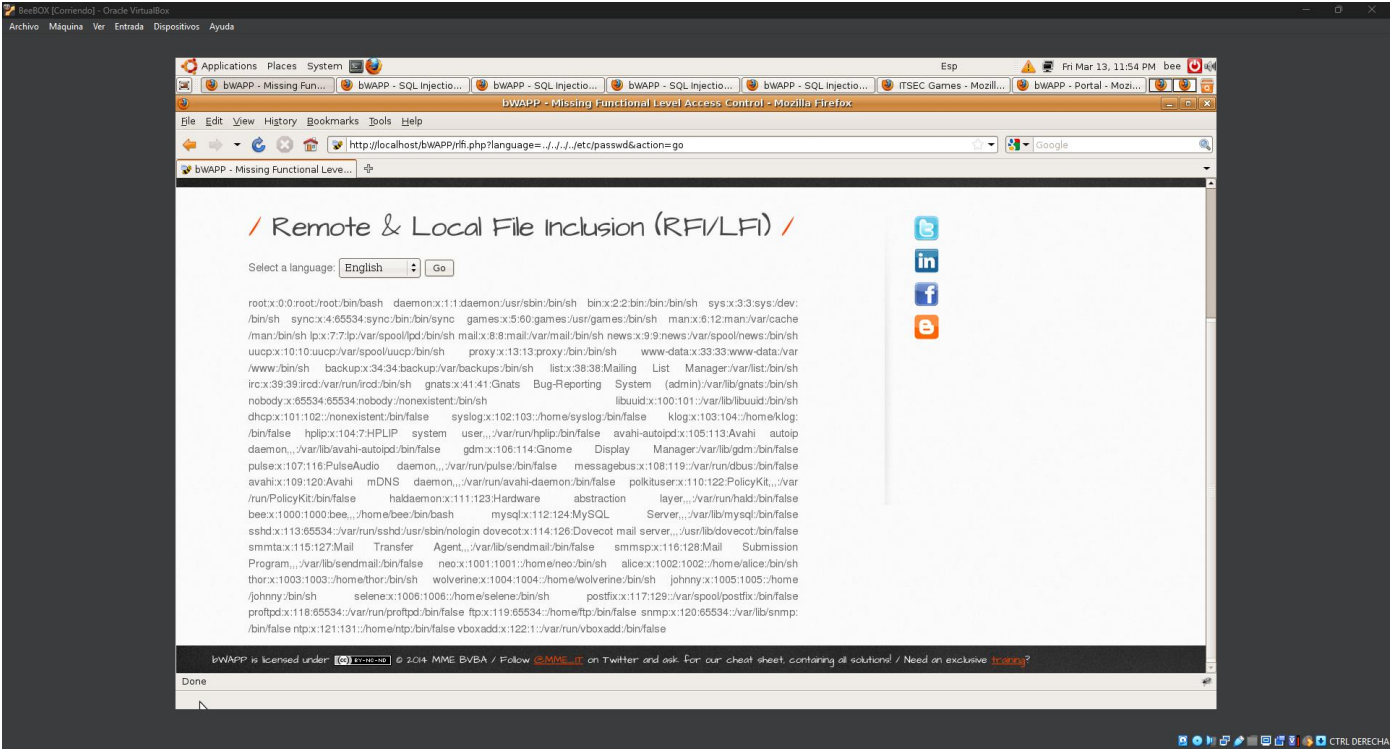
Descripción

LFI permite a un atacante acceder a archivos del sistema.

Procedimiento

- Se modificó el parámetro **language** de la URL.

`../../../../etc/passwd`



Explicación técnica

El servidor incluye archivos basándose en parámetros de la URL sin validación.

Riesgos

- Exposición de archivos del sistema.
- Obtención de información sensible.

Mitigación

- Validar rutas de archivos.
- Usar listas blancas de archivos permitidos.

4.6 Server Side Request Forgery (SSRF)

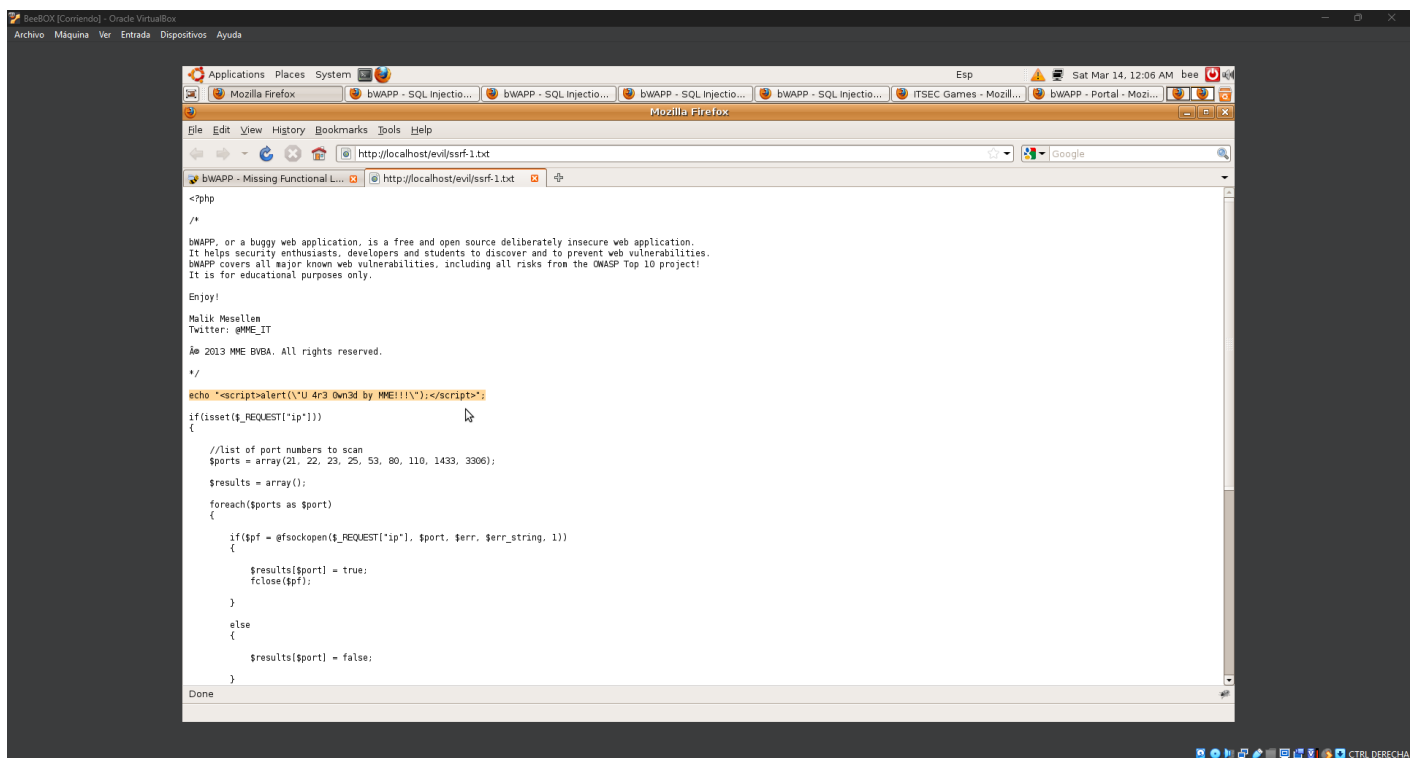
Descripción

SSRF permite a un atacante hacer que el servidor realice solicitudes HTTP a recursos internos.

Procedimiento

- Se introdujo una URL interna:

```
http://localhost/evil/ssrf-1.txt
```



Explicación técnica

El servidor realiza solicitudes sin validar el destino.

Riesgos

- Acceso a servicios internos.
- Escaneo de red interna.

Mitigación

- Validar URLs externas.
- Bloquear acceso a redes internas.

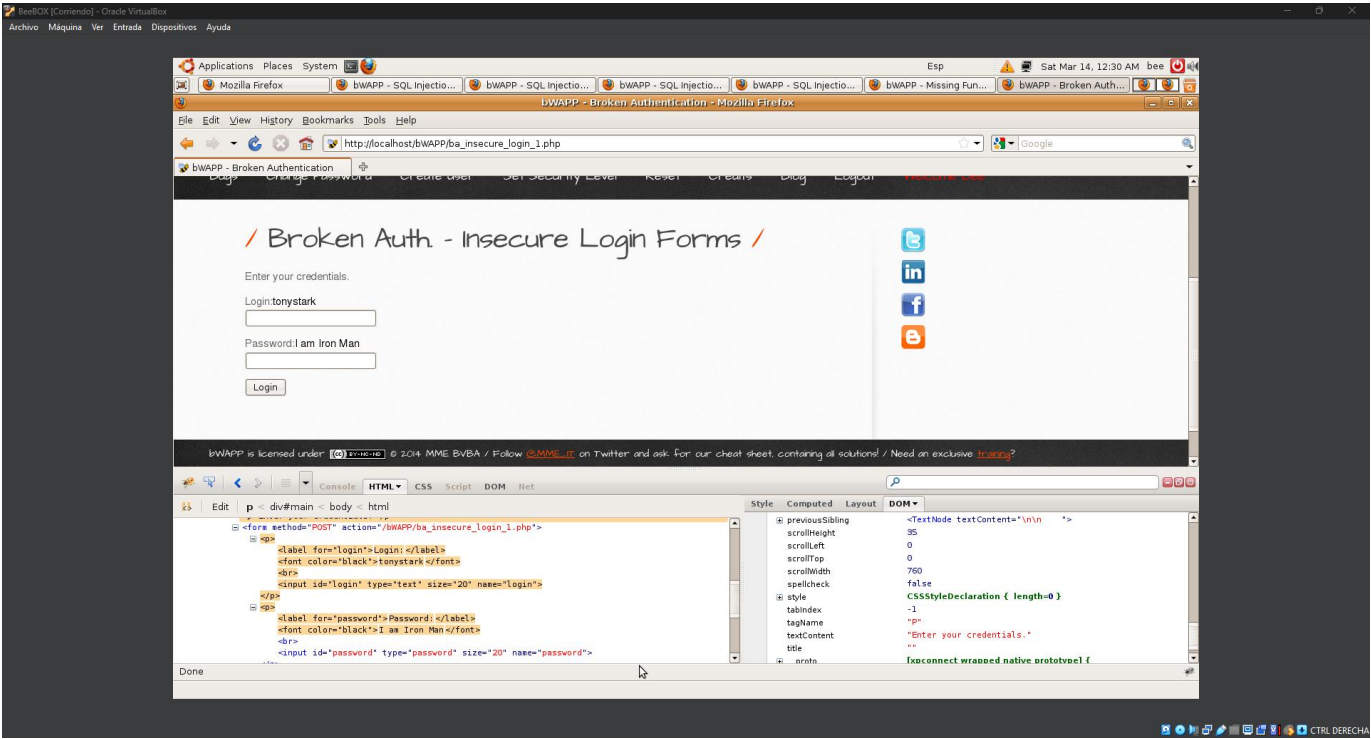
4.7 Insecure Design

Descripción

Insecure Design ocurre cuando el sistema presenta fallos de seguridad desde su diseño.

Procedimiento

- Se inspeccionó el formulario de login y se visualizaron credenciales ocultas.



Riesgos

- Acceso directo al sistema.
- Falta de protección de datos sensibles.

Mitigación

- Diseño seguro desde la fase de desarrollo.
- Implementación de controles de autenticación robustos.

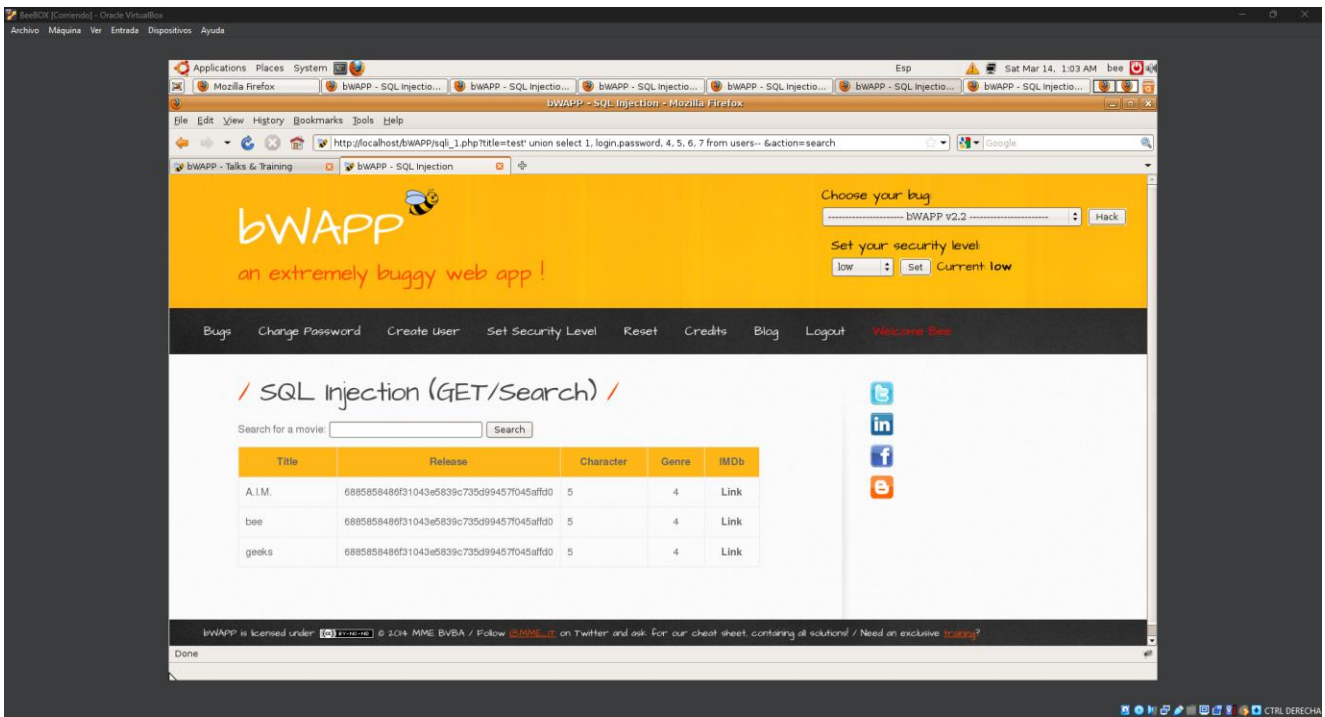
4.8 Weak Password Hashing

Descripción

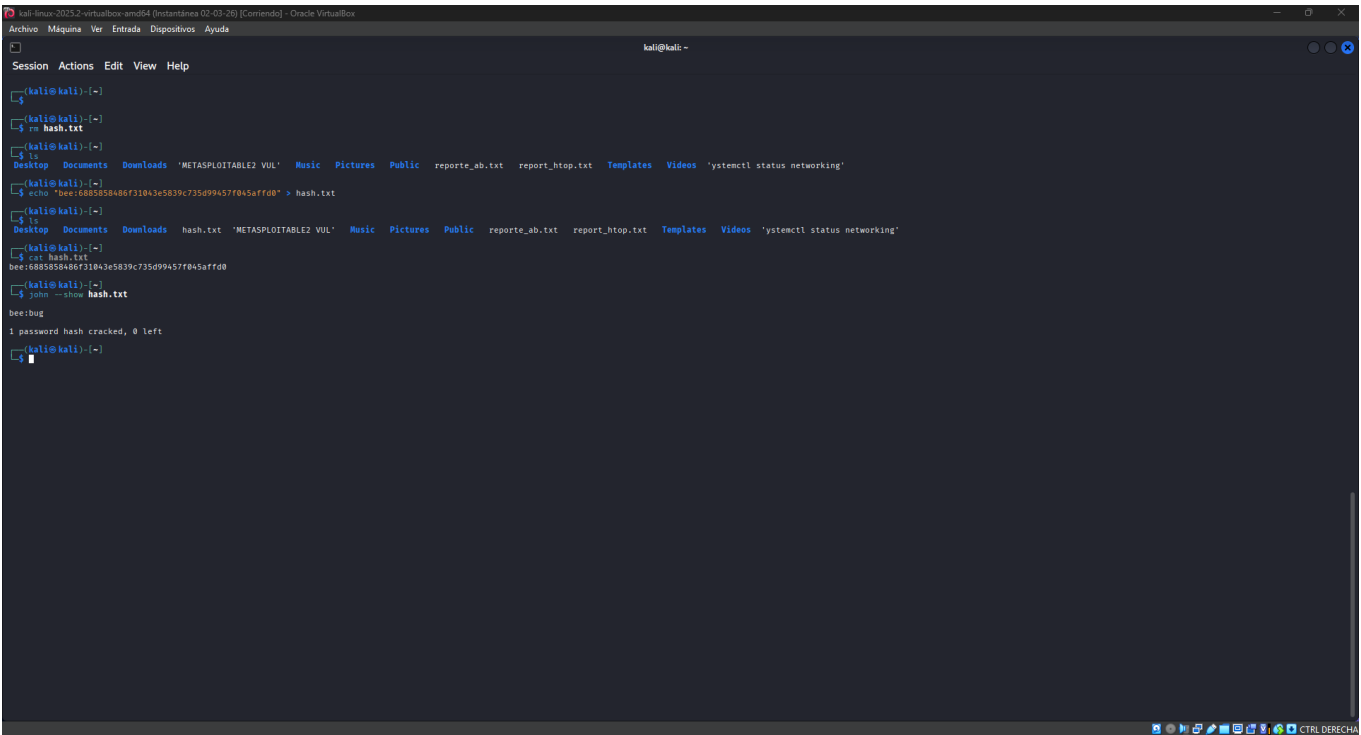
El uso de algoritmos de hash débiles permite crackear contraseñas fácilmente.

Procedimiento

Se extrajeron hashes mediante SQL Injection.



- Posteriormente se utilizó **John the Ripper** para crackearlos.



Riesgos

- Compromiso de contraseñas.
- Acceso no autorizado a cuentas.

Mitigación

- Uso de **bcrypt**, **Argon2** o **PBKDF2**.
- Implementar salting.

4.9 Security Logging and Monitoring Failures

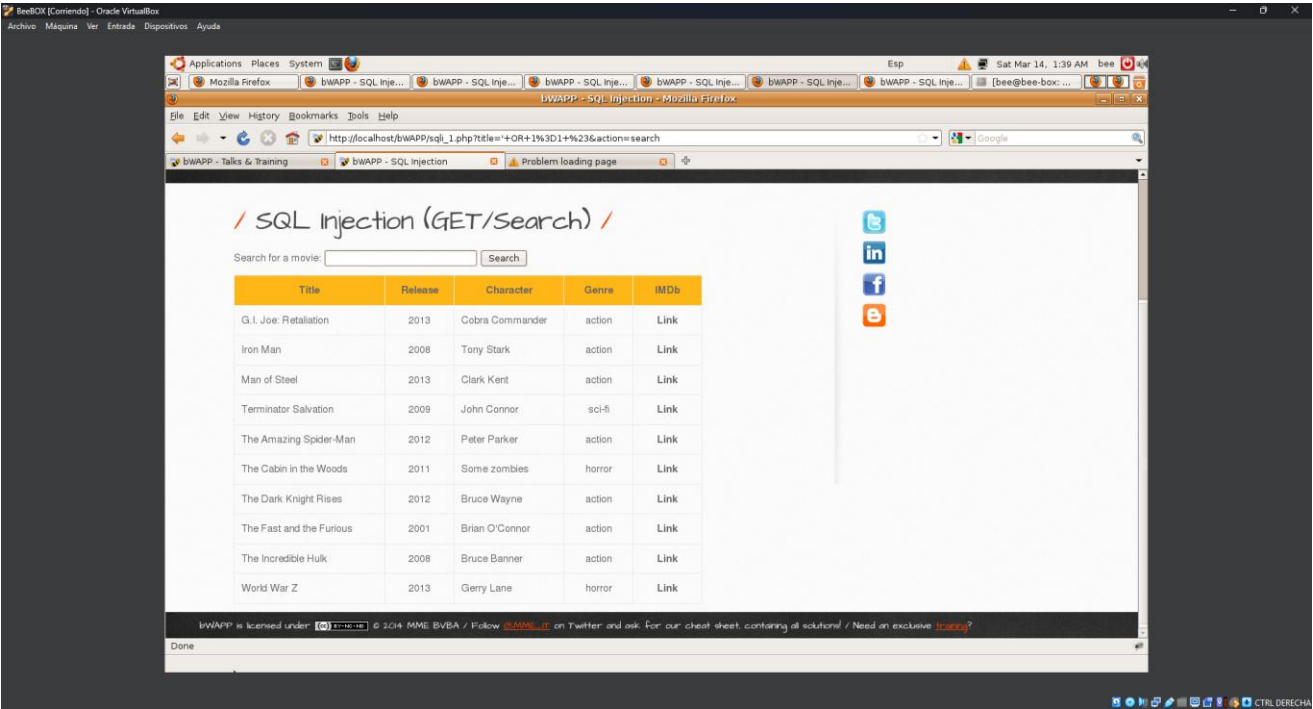
Descripción

Se produce cuando la aplicación no registra o monitoriza correctamente eventos de seguridad.

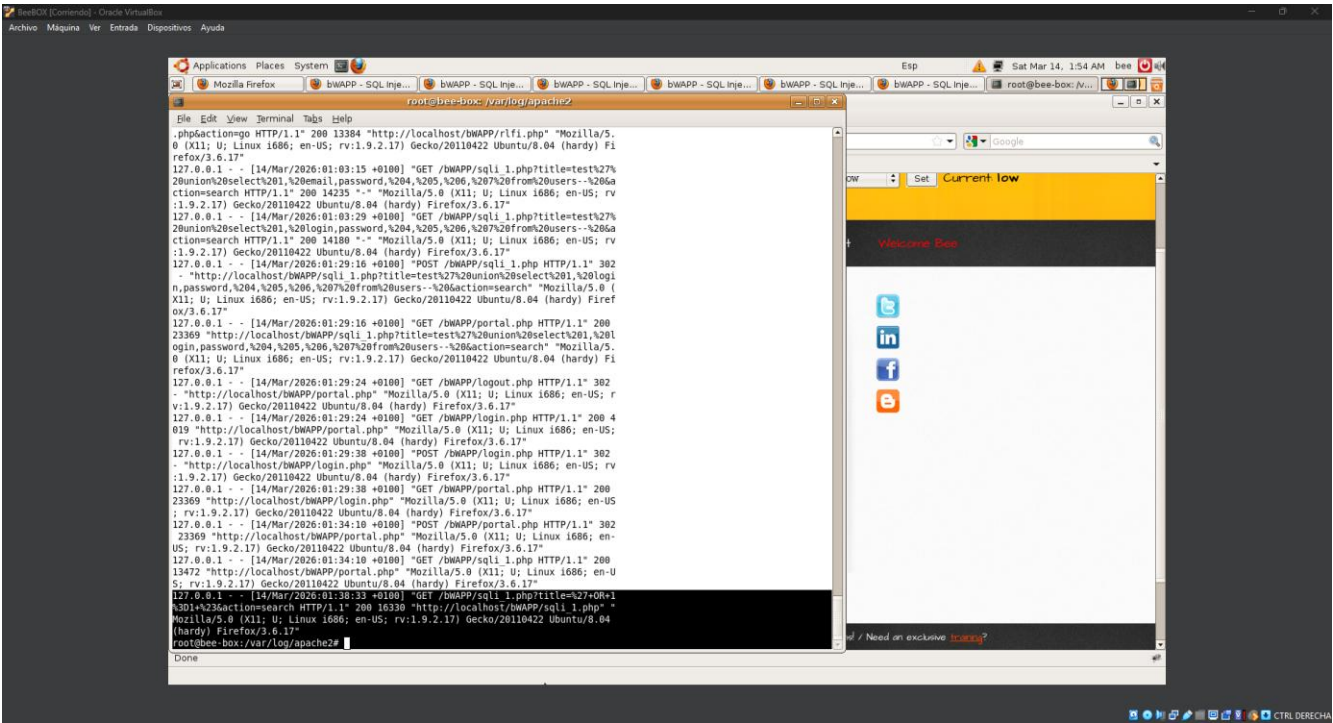
Procedimiento

- Se realizó una inyección SQL.

' OR 1=1 #



- Posteriormente se revisaron los logs del servidor.



Riesgos

- Ataques sin detección.
- Dificultad para responder a incidentes.

Mitigación

- Implementar sistemas de logging adecuados.
- Monitorizar eventos de seguridad.

4.10 Vulnerable and Outdated Components

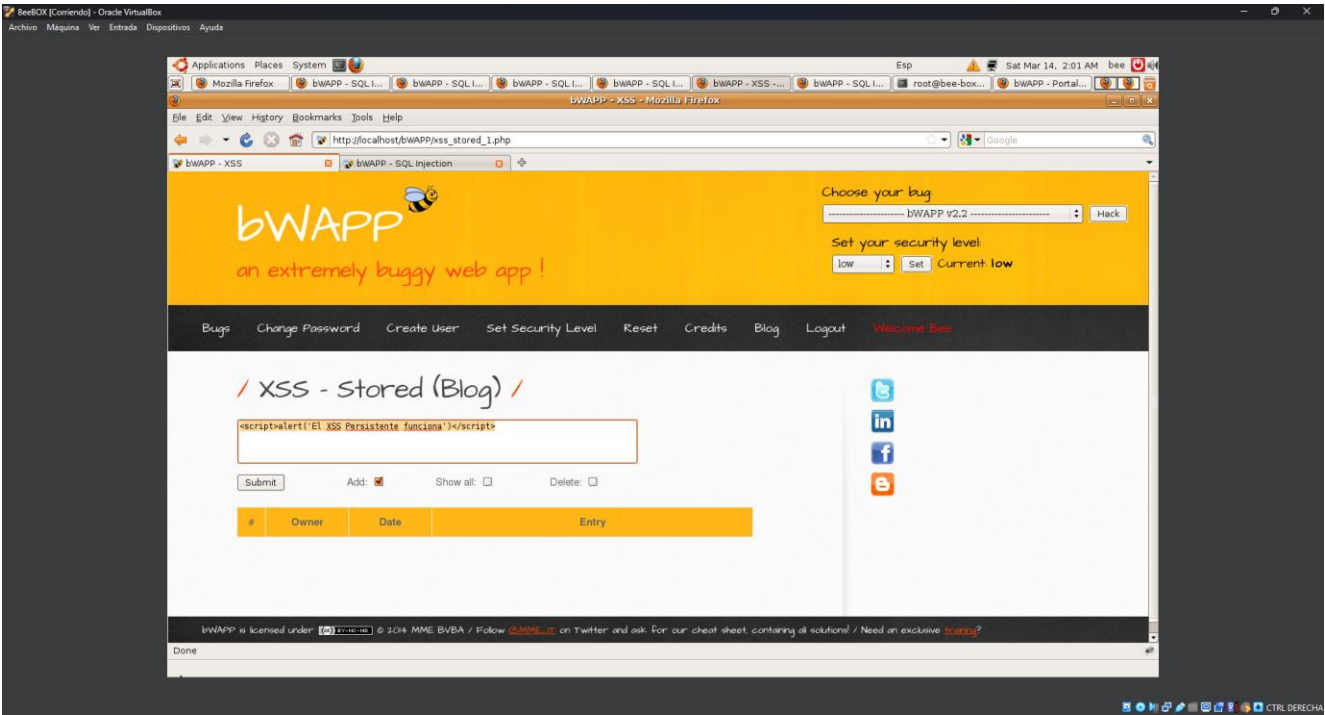
Descripción

El uso de software desactualizado puede introducir vulnerabilidades conocidas.

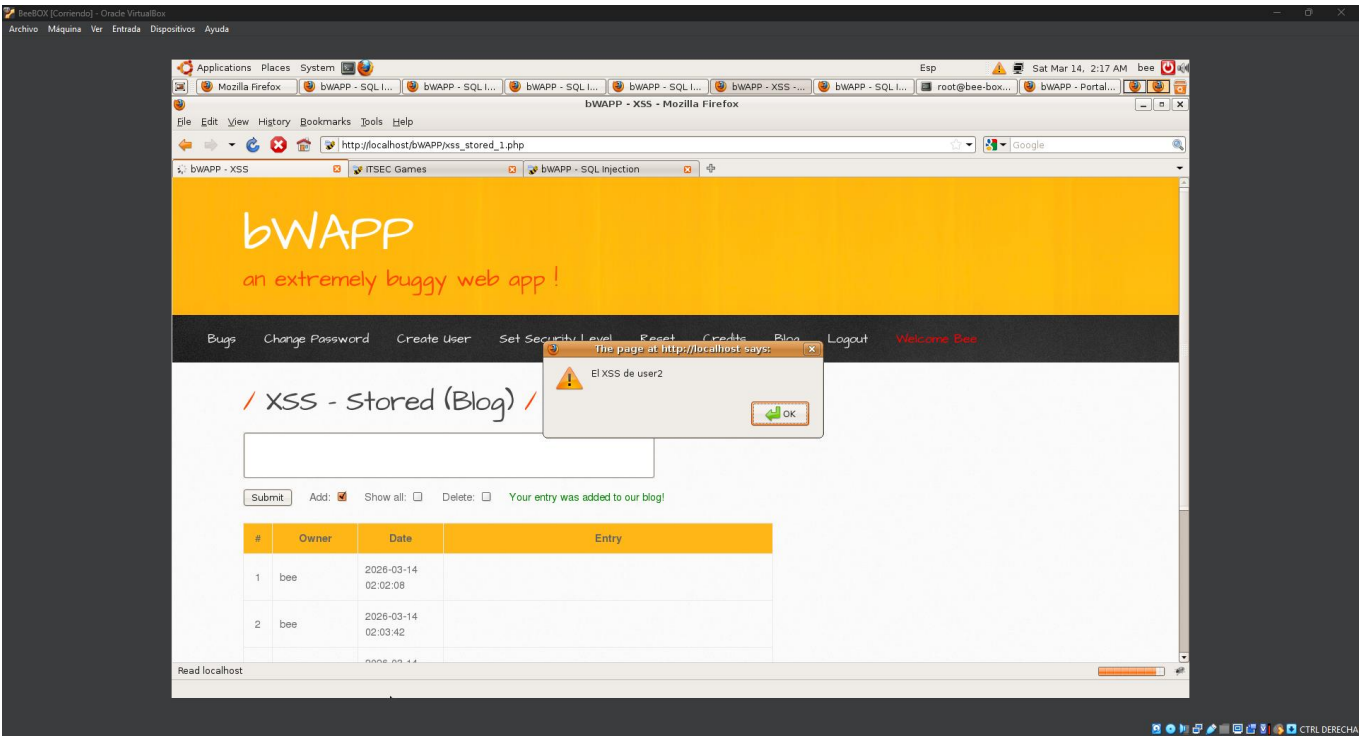
Procedimiento

- Se utilizó la funcionalidad **Stored Blog** para inyectar código JavaScript persistente.

`<script>alert('XSS Persistente')</script>`



- Al recargar la página el script se ejecutó.



Riesgos

- Ataques persistentes a múltiples usuarios.
- Robo de información.

Mitigación

- Mantener librerías actualizadas.
- Validar y sanitizar entradas.

5. Conclusiones

Durante esta práctica se han analizado diversas vulnerabilidades incluidas en el **OWASP Top 10**, utilizando la aplicación vulnerable bWAPP como entorno de laboratorio.

El ejercicio ha permitido comprender cómo errores en el diseño, desarrollo o configuración de una aplicación web pueden dar lugar a vulnerabilidades críticas que pueden ser explotadas por atacantes. Estas vulnerabilidades pueden comprometer la confidencialidad, integridad y disponibilidad de los sistemas.

Además, se ha podido observar la importancia de implementar mecanismos de seguridad adecuados, tales como validación de entradas, control de accesos, uso de algoritmos criptográficos seguros y monitorización de eventos de seguridad.

Finalmente, esta práctica demuestra que la seguridad debe ser considerada desde las primeras fases del desarrollo de una aplicación, aplicando buenas prácticas de programación segura y manteniendo los sistemas actualizados para reducir el riesgo de ataques.

FIN